

Wheat Water Productivity Dashboard

Project overview, setup and API reference

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II
Generated 2026-07-29

Wheat Water Productivity (WWP) Dashboard

Interactive web dashboard that assesses and monitors wheat water productivity in Ethiopia from FAO WaPOR v3 satellite data, with a LightGBM prediction service and per-prediction explanations.

Built for the Ethiopian Institute of Agricultural Research (EIAR) under the WaPOR Phase II project, developed by IWMI East Africa with FAO, supported by the Government of the Netherlands. Implements the deliverables in [ToR_Dashboard_Developer.docx](#) ; see the [ToR Coverage document](#) for the work-package mapping and the [Architecture document](#) for the system design.

What it does

Pick an area of interest three ways — an administrative unit (region → zone → woreda), an uploaded field or scheme boundary (zipped shapefile or GeoJSON), or a polygon drawn on the map. Choose a production system (rainfed or irrigated) and a season, then run the analysis. The backend retrieves seasonal NPP, assembles ten biophysical and socioeconomic features, runs LightGBM inference over the extent, and returns a 100 m water-productivity raster with zonal statistics, a class distribution, a five-season trend, and model feature importance. Click any pixel for its value, then **Explain this prediction** to see each feature's signed contribution to that specific prediction.

Requirements

- Python 3.10+
- Node.js 18+

Quick start

```
# 1. Backend - installs deps, trains the initial model on first run
cd backend
pip install -r requirements.txt
python train_model.py          # writes models/v0001/, prints holdout metrics
uvicorn app.main:app --port 8000

# 2. Frontend - in a second terminal
cd frontend
npm install
npm run dev                    # http://localhost:5173, proxies /api to :8000
```

For a single-process deployment, build the frontend and let the backend serve it:

```
cd frontend && npm run build    # emits frontend/dist
cd ../backend && uvicorn app.main:app --port 8000
# dashboard at http://localhost:8000, API at http://localhost:8000/api
```

`npm install` may report that esbuild's install script was skipped. If the build then fails, run `npm approve-scripts esbuild` once.

Project layout

```
backend/  
  app/  
    main.py          FastAPI app; mounts /api and serves frontend/dist  
    api.py           REST routes, request validation  
    engine.py        WWPT analytical engine: grid, raster, statistics, export  
    model_service.py LightGBM training, versioned artifacts, inference, explain  
    geodata.py       WaPOR data-access layer + explanatory-feature assembly  
    aoi.py           AOI resolution, shapefile/GeoJSON validation, geometry  
    cache.py         TTL response cache + idempotency-key store  
    pnglib.py        Dependency-free RGBA PNG encoder for the result raster  
  models/           Versioned model artifacts (generated; vNNNN/ + current.json)  
  train_model.py     CLI to train or inspect the active model  
frontend/  
  src/  
    App.jsx          Dashboard shell, state, user journeys  
    MapView.jsx      Leaflet map: AOI, raster overlay, drawing, pixel inspect  
    charts.jsx       Inline-SVG charts with hover tooltips and table views  
    content.jsx      Methodology, data sources, user guide, citation, disclaimer  
    api.js           Backend client with typed error messages  
    styles.css       EIAR visual identity and layout  
  tests/  
    test_api_e2e.py  47 API checks: journeys, caching, validation, errors  
    test_retrain.py  Model retrain-and-promote workflow  
    test_ui.mjs      51 browser checks: rendering, interaction, responsive layout  
  docs/  
    ARCHITECTURE.md  System design, data flow, API reference, deployment  
    TOR_COVERAGE.md  Work-package coverage and what deployment still needs
```

API

All routes are under `/api`. Interactive documentation is generated at `/docs` when the server is running.

METHOD	ROUTE	PURPOSE
GET	/health	Liveness plus active model version
GET	/admin-units	Region/zone/woreda tree, valid years and seasons
POST	/upload	Validate a boundary file, return its id, bounds and GeoJSON
POST	/analysis	Run an analysis over an AOI; honours Idempotency-Key
GET	/predict	WWP at a single point
GET	/explain	Per-feature contributions for one prediction
GET	/export/csv	Per-cell values for a completed run
GET	/model/info	Active version, holdout metrics, feature importance
POST	/model/retrain	Train a candidate on new observations; promote if not worse

Tests

Start the backend first, then:

```
python tests/test_api_e2e.py      # 47 API checks
python tests/test_retrain.py      # retrain-and-promote workflow

cd tests
npm install playwright && npx playwright install chromium
node test_ui.mjs                  # 51 browser checks, writes screenshots/
```

Point any suite at a different host with `WWP_BASE=http://host:port` .

Data sources

Analysis values come from a **synthetic data provider** (`geodata.SyntheticProvider`) that generates deterministic, spatially coherent fields, so the full pipeline runs end-to-end without WaPOR credentials. It is a drop-in interface: implementing `assemble()` against the live FAO WaPOR v3 retrieval and the EthioSIS / SRTM / survey layers switches the system to real data without changes anywhere else. See [the Architecture document § Data-access layer](#).